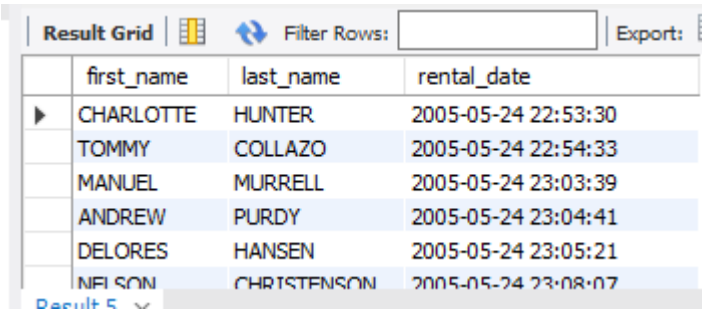
ASSIGNMENT 10 – SQL

Sakila Database

Q1. Write a query to display the first name, last name, and rental date of all customers who rented a film. Use the customer and rental tables.

```
select  c.first_name, c.last_name, r.rental_date
from customer c
inner join rental r
on c.customer_id = r.customer_id
order by r.rental_date, c.last_name, c.first_name ;
```

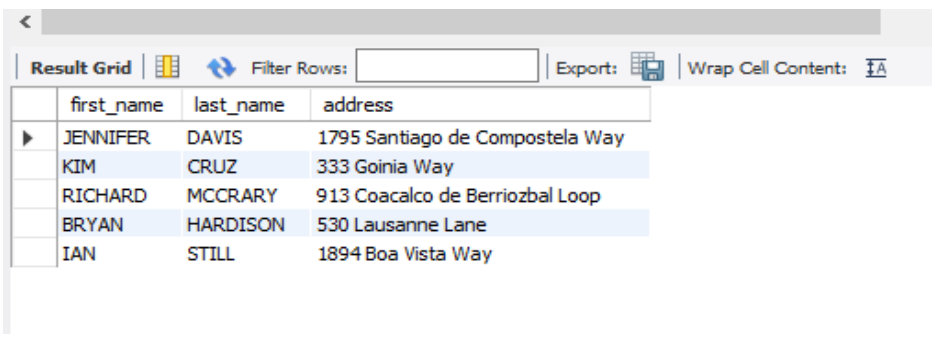


Result Grid | Filter Rows: | Export:

first_name	last_name	rental_date
CHARLOTTE	HUNTER	2005-05-24 22:53:30
TOMMY	COLLAZO	2005-05-24 22:54:33
MANUEL	MURRELL	2005-05-24 23:03:39
ANDREW	PURDY	2005-05-24 23:04:41
DELORES	HANSEN	2005-05-24 23:05:21
NELSON	CHRISTENSON	2005-05-24 23:08:07

Q2. Write a query to display the first name, last name, and address of all customers whose district is either Alberta or Texas. Use the customer and address tables.

```
select  c.first_name, c.last_name, d.address
from customer c
left join address d
on c.address_id = d.address_id
where district = "Alberta" or district = "Texas";
```

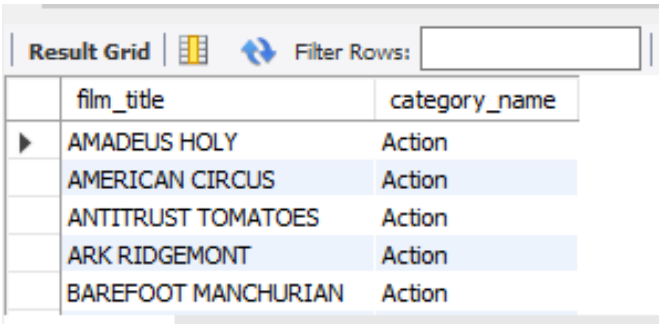


Result Grid | Filter Rows: | Export: | Wrap Cell Content:

first_name	last_name	address
JENNIFER	DAVIS	1795 Santiago de Compostela Way
KIM	CRUZ	333 Goinia Way
RICHARD	MCCRARY	913 Coacalco de Berriozbal Loop
BRYAN	HARDISON	530 Lausanne Lane
IAN	STILL	1894 Boa Vista Way

Q3. List all films along with their category names, including films that may not belong to any category. Then sort it by category names. Use the film, film_category and category tables.

```
select f.title as film_title, c.name as category_name
from film f
left join film_category fc
on f.film_id = fc.film_id
left join category c
on fc.category_id = c.category_id
order by c.name,f.title;
```

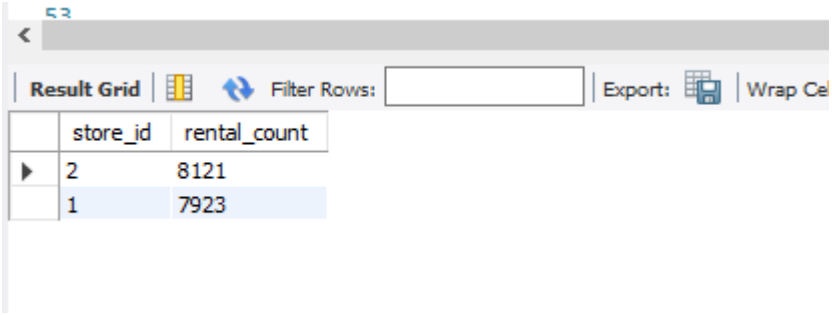


Result Grid | Filter Rows:

film_title	category_name
AMADEUS HOLY	Action
AMERICAN CIRCUS	Action
ANTITRUST TOMATOES	Action
ARK RIDGEMONT	Action
BAREFOOT MANCHURIAN	Action

Q4. Display the number of rentals per store. Use the rental, inventory, and store tables.

```
select s.store_id, count(r.rental_id) as rental_count
from store s
inner join inventory i
on s.store_id = i.store_id
inner join rental r
on i.inventory_id = r.inventory_id
group by s.store_id
order by rental_count Desc;
```

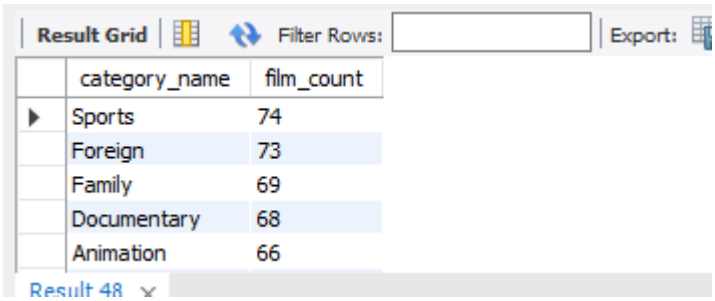


Result Grid | Filter Rows: | Export: | Wrap Ce

store_id	rental_count
2	8121
1	7923

Q5. List the names of categories that have more than 50 films. Use table category and film_category.

```
select c.name as category_name,count(fc.film_id) as film_count
from category c
inner join film_category fc
on c.category_id = fc.category_id
group by c.category_id, c.name
having count(fc.film_id) > 50
order by film_count desc;
```



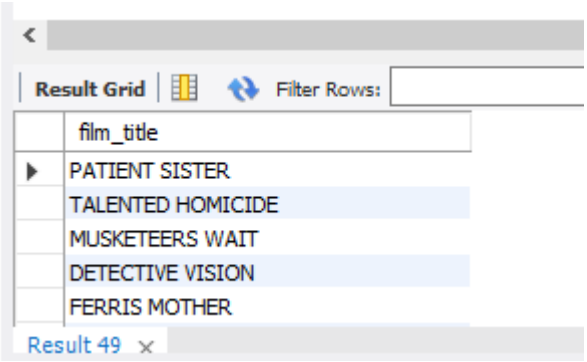
Result Grid | Filter Rows: | Export:

category_name	film_count
Sports	74
Foreign	73
Family	69
Documentary	68
Animation	66

Result 48

Q6. Write a query to display the film titles that have been rented by the customer named 'Mary Smith'. Use a subquery to get the customer ID.

```
select f.title as film_title
from film f
inner join inventory i
on f.film_id = i.film_id
inner join rental r
on i.inventory_id = r.inventory_id
where r.customer_id = (select c.customer_id from customer c where c.first_name =
'Mary' and c.last_name = 'Smith');
```



Result Grid | Filter Rows:

film_title
PATIENT SISTER
TALENTED HOMICIDE
MUSKETEERS WAIT
DETECTIVE VISION
FERRIS MOTHER

Result 49

Q7. Write a query to display the total rental income generated by each film. Use film, inventory, rental and payment tables.

```
select f.title as film_title,sum(p.amount) as total_rental_income
from film f
inner join inventory i
on f.film_id = i.film_id
inner join rental r
on i.inventory_id = r.inventory_id
inner join payment p
on r.rental_id = p.rental_id
group by f.film_id, f.title
order by total_rental_income Desc;
```

film_title	total_rental_income
TELEGRAPH VOYAGE	231.73
WIFE TURN	223.69
ZORRO ARK	214.69
GOODFELLAS SALUTE	209.69
SATURDAY LAMBS	204.72
TITANS JERK	201.71
TORQUE BOUND	198.72
HARRY IDAHO	195.70

Q8. Write a query to display the total number of films each actor has acted in. Use actor, film_Actor tables.

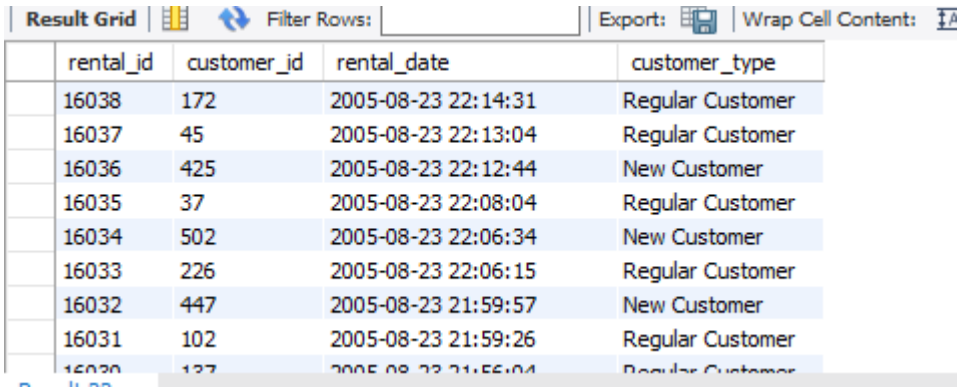
```
select a.first_name ,a.last_name ,count(fa.film_id) as total_films
from actor a
inner join film_actor fa
on a.actor_id = fa.actor_id
group by a.actor_id, a.first_name, a.last_name
order by total_films desc, a.last_name, a.first_name;
```

Result Grid | Filter Rows: | Export: | Wrap

first_name	last_name	total_films
GINA	DEGENERES	42
WALTER	TORN	41
MARY	KEITEL	40
MATTHEW	CARREY	39
SANDRA	KILMER	37
SCARLETT	DAMON	36
VIVIEN	BASINGER	35
HENRY	BERRY	35

Q9. Write a query to list all rentals with a case statement that displays 'New Customer' if the customer_id is greater than 300, and 'Regular Customer' otherwise. Use table rental.

```
select rental_id,customer_id,rental_date,
case when  customer_id > 300 then 'New Customer'
else 'Regular Customer'
end as customer_type
from rental
order by rental_date desc, customer_id;
```

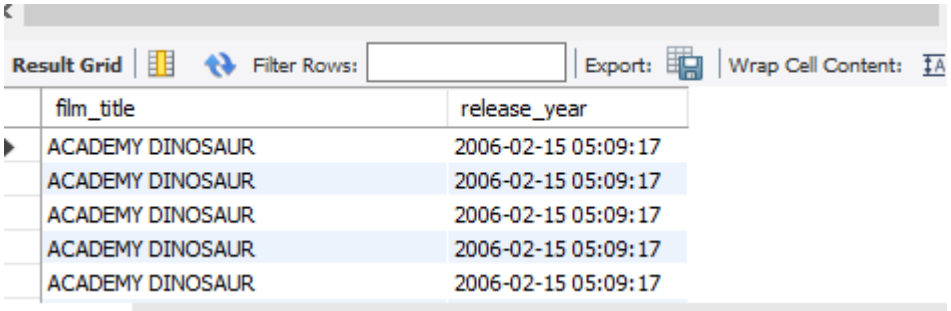


Result Grid | Filter Rows: | Export: | Wrap Cell Content:

rental_id	customer_id	rental_date	customer_type
16038	172	2005-08-23 22:14:31	Regular Customer
16037	45	2005-08-23 22:13:04	Regular Customer
16036	425	2005-08-23 22:12:44	New Customer
16035	37	2005-08-23 22:08:04	Regular Customer
16034	502	2005-08-23 22:06:34	New Customer
16033	226	2005-08-23 22:06:15	Regular Customer
16032	447	2005-08-23 21:59:57	New Customer
16031	102	2005-08-23 21:59:26	Regular Customer

Q10. Write a query to find all films with titles that start with the letter 'A'. Display the film title and the release year. Use film table and inventory table.

```
select f.title as film_title, i.last_update as release_year
from film f
inner join inventory i
on f.film_id = i.film_id
where f.title like "A%";
```



Result Grid | Filter Rows: | Export: | Wrap Cell Content:

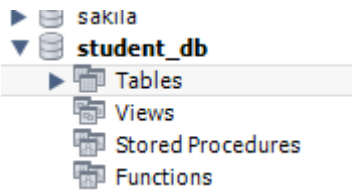
film_title	release_year
ACADEMY DINOSAUR	2006-02-15 05:09:17
ACADEMY DINOSAUR	2006-02-15 05:09:17
ACADEMY DINOSAUR	2006-02-15 05:09:17
ACADEMY DINOSAUR	2006-02-15 05:09:17
ACADEMY DINOSAUR	2006-02-15 05:09:17

Q11. Create a new DB and perform below operations using SQL commands.

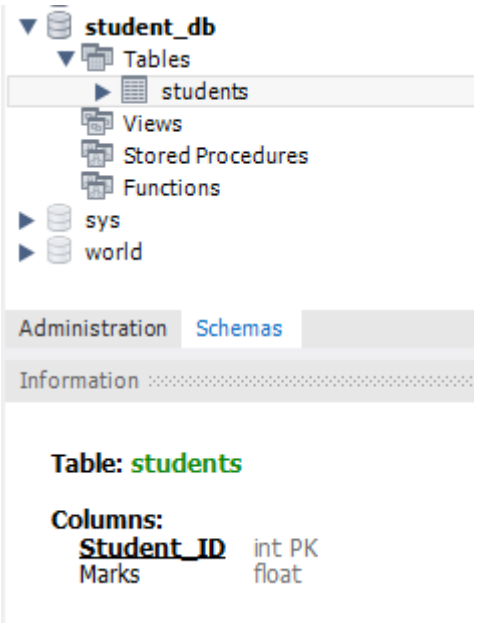
(a) Create a table with 2 columns – Student_ID, Marks

```
create database Student_DB;
```


```
use Student_DB;
```



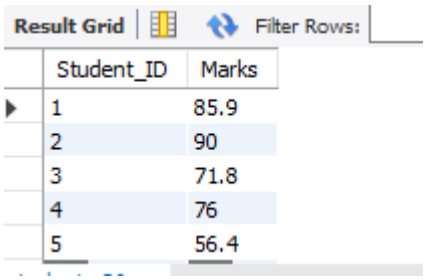

```
create table Students (
    Student_ID int primary key,
    Marks float
);
```



(b) Insert 2 records in this table.

```
insert into Students (Student_ID, Marks)
values (1, 85.9),(2, 90),(3, 71.8),(4, 76),(5, 56.4);

select * from students;
```

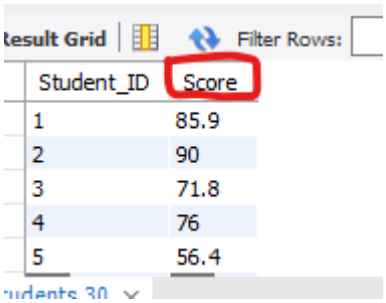


Result Grid | Filter Rows:

Student_ID	Marks
1	85.9
2	90
3	71.8
4	76
5	56.4

(c) Rename the Marks column with Score.

```
alter table Students
change column Marks Score float;
```



Result Grid | Filter Rows:

Student_ID	Score
1	85.9
2	90
3	71.8
4	76
5	56.4

(d) Then delete the table.

```
drop table Students;
```

46	10:10:01	DROP TABLE Students	0 row(s) affected
47	10:10:12	select * from students LIMIT 0, 1000	Error Code: 1146. Table 'student_db.students' doesn't exist

(e) Then delete the Database.

```
drop database Student_DB;
```

48	10:12:22	DROP DATABASE Student_DB	0 row(s) affected